NMAM INSTITUTE OF TECHNOLOGY, NITTE

Department of Information Science and Engineering

OPERATING SYSTEMS

PROGRAMMING ASSIGNMENT REPORT

Readers-Writers Problem and Dining Philosophers Problem

(Semaphore-based and Monitor-based implementations in Python)

Submitted By

Name: Disha Shetty
USN: NNM24IS077
GitHub: https://github.com/Disha118/OS-Synchronization-Assignment

Submitted to

Dr Jason Elroy Martis
Associate Professor
Dept Information Science and Engineering, Nitte

1. Problem Statement and Objective

Process synchronization is an integral component of the OS, which ensures coordination and safety of operation of different threads when using the resources. The race conditions, deadlocks, and starvation are possible, when multiple threads use the same resources without using synchronization.

The task involves solving the two well-known synchronization problems Readers-Writers and Dining Philosophers using semaphores and monitors. The Readers-Writers problem includes several readers using shared data simultaneously and a writer who requires exclusive access for modification of this data. The Dining Philosophers problem includes four philosophers and four forks that should be coordinated by philosophers.

The goal of this task is to investigate how semaphores and monitors help in managing of the shared resources and coordination of the threads. Python threading, locks, and condition variables are used in order to control the resources and prevent the synchronization problems. The events and thread state transitions are recorded and then used to create HTML simulation of the process synchronization.

By performing this task, we will see the difference between semaphore and monitor approach and the way of interaction of threads in the process of their execution.

2. Readers-Writers - Semaphore Solution

Variable database_value with an initial value 100 is introduced in order to show the database value. Write operations are counted through variable version, while the number of readers is shown by means of reader_count.

Classes threading.Semaphore and threading.Lock from the Python standard library are used in order to protect access to shared resources. Resource semaphore is responsible for managing access to the database, reader_count_lock for updating reader_count, and state_lock – for execution-state protection and events recording.

The code creates six reader threads and three writer threads. Reader threads read the database three times, while writers write into the database twice. Several readers may read the database at once; however, writers require exclusive access to it. Writers increase the database value by the integer from 5 to 20; hence, the value of version must be equal to 6.

The code implements readers-preference approach that provides access to the database for several readers if no writers use it. However, the writers will experience some delays if readers continually use the resource.

The code records all events occurred during the process of the program execution, including reading/writing operations, semaphore operations and end of the threads execution. These events are

3. Readers-Writers - Monitor Solution

Common variables such as active_readers, active_writer, waiting_readers and waiting_writers are used for tracking of threads activity. Database_value variable is equal to 100 and it contains a value of the database. The variable version tracks a successful write operation.

In order to manage access to the shared database a monitor is implemented through the threading.Condition function provided by Python. Many readers can simultaneously perform their operation but writers have exclusive access.

Six reader threads and three writer threads are generated. Read operation is performed by readers thrice while writers perform their operations twice. Each writer increases the value of the database by a random number between 5 and 20, that is why database_version becomes equal to 6.

The writer priority policy is used in this case; readers wait in case there is an active or waiting writer. The monitor.wait() and monitor.notify_all() functions are used for threads to wait and resume their work after some changes in access conditions.

Execution events are tracked and used to generate the file readers_writers_monitor.html that provides visualization of database state, threads activity and execution sequence.

4. Dining Philosophers - Semaphore Solution

Number of philosophers equals to four, number of forks equals to four; philosopher needs two forks for eating. Number of semaphores equals to four: fork semaphores for regulation of access to certain fork and room semaphore for regulation of number of philosophers that try to obtain forks at the same time not more than three philosophers and avoiding deadlock.

Threading.Lock is used for synchronization of all shared data such as state of philosophers, fork's owner, number of meals. Philosophers must perform the following operations: thinking, getting hungry, taking the left and right forks, eating, releasing the left and right forks.

Each philosopher eats two times; therefore, in total there will be eight meals. Room semaphore does not let all philosophers compete for forks at once in order to avoid deadlock. Fork semaphores do not let two philosophers use the same fork at once.

During the run of the program there is recording of different events: thinking, requesting of forks, taking/leasing of forks, eating, finishing of meal. All those events are used to create a file dining_philosophers_semaphore.html for displaying current state of philosophers, available forks, number of meals and flow of execution.

5. Dining Philosophers - Monitor Solution

Four philosophers and four forks are used in order to solve the problem of Dining Philosophers using a monitor. The shared variables states, fork_owner and meals_completed represent the state, the ownership and the number of meals, which were eaten by each philosopher.

Python's Threading.Condition class is used in order to create a monitor and control the access to the shared variables. The can_eat() method verifies whether the neighbors of the philosopher are currently eating. If yes, then the philosopher waits using the method monitor.wait(). If the philosopher can perform something, he takes forks and starts to eat.

Each philosopher eats two times, that is why there are eight meals in the whole program. When the philosopher finishes his meal, he releases forks and calls the method monitor.notify_all() in order to inform other philosophers about the availability of the resources.

All the actions, which take place during the execution of the program are registered by it. It includes thinking, waiting, taking forks, eating, releasing forks, finishing meals and so on. The information is used for creating dining_philosophers_monitor.html file, where the state of the philosophers, the availability of forks and so on are provided.

6. Event Recording and HTML Generation

1. Readers-Writers Problem Using Semaphores

The recording of the activities that the readers and writers are engaging in during their access to the database is done using this program. This will be activities such as data reading, data updating, acquiring of resources, releasing of resources, and completion of activity. Logging of activities will enable us to understand how semaphores and locks work during database access.

Logging of activities is used to generate readers_writers_semaphore.html file. The HTML file contains details of database values, version numbers, readers and writers' activities, and activities' execution order. This is very useful in understanding how readers and writers access the database.

2. Readers-Writers Problem Using Monitors

The log records events performed by readers and writers, for instance, the amount of running and waiting threads and the moments when the database was updated. This information describes how the monitor manages access to the database and treats waiting writers.

The gathered information is used to generate readers_writers_monitor.html. This document represents the state of the database, activities of threads, and the history of their execution. It helps to understand the algorithm of control of the monitor and how it prevents simultaneous reading and writing.

3. Dining Philosophers Problem Using Semaphores

The program logs activities like thinking, asking for forks, taking them, eating, and releasing them of each of the philosophers. In addition, the program logs the availability of forks and the number of meals served. This information will be useful in demonstrating how semaphores are used in the coordination of forks and deadlock prevention.

The events will be used to create the dining_philosophers_semaphore.html page. The page contains the status of the philosophers, the status of the forks, the meals served, and execution log of each philosopher.

4. Dining Philosophers Problem Using Monitors

This process observes the actions carried out by the philosopher like thinking, waiting for the fork, eating, releasing the fork, and having dinner. It also observes the state of the philosophers and the

owner of the fork. In this way, we can be able to see how the monitor facilitates access to the fork that is common to all.

The events are used to create the file known as dining_philosophers_monitor.html. In this webpage, there is information concerning the states of the philosophers, whether the fork is available, the process of having dinner, and the execution process. The controls help in carrying out the execution process.

7. Execution Traces

7.1 Readers-Writers Using Semaphore

Event **36** · 0s
Reader 3 starts reading operation 1. Multiple readers may read together.

Event **37** · 0.168s
Reader 2 completed operation 1: read database value 100, version 0.

Event **38** · 0.168s
Reader 2 finished reading. Reader count is now 5.

Event **39** · 0.193s
Reader 3 completed operation 1: read database value 100, version 0.

Event **40** · 0.193s
Reader 3 finished reading. Reader count is now 4.

The trace records synchronization events and shows how readers and writers access the shared database without conflicts.

7.2 Readers-Writers Using Monitor

Event **25** · 0.001s
Reader 6 waits because a writer is active or waiting. Condition.wait() releases the monitor lock while waiting.

Event **26** · 0.001s
Reader 1 thread created and ready.

Event **27** · 0.001s
Reader 1, operation 1: requests permission to read.

Event **28** · 0.001s
Reader 1 waits because a writer is active or waiting. Condition.wait() releases the monitor lock while waiting.

Event **29** · 0.401s
Reader 4, operation 1: read value 100 at database version 0.

Event **30** · 0.401s
Reader 4 leaves the reading section. Active readers remaining: 1.

Event **31** · 0.411s
Reader 3, operation 1: read value 100 at database version 0.

The event history shows how the monitor coordinates reader and writer access using condition variables and lock management.

7.3 Dining Philosophers Using Semaphore

Event	**1**	·	0s
Simulation started. Semaphore objects are ready.			
Event	**2**	·	0s
Philosopher 1 is thinking before meal 1.			
Event	**3**	·	0s
Philosopher 3 is thinking before meal 1.			
Event	**4**	·	0s
Philosopher 2 is thinking before meal 1.			
Event	**5**	·	0s
Philosopher 4 is thinking before meal 1.			
Event	**6**	·	0.48s
Philosopher 2 is hungry and requests forks 2 and 3.			
Event	**7**	·	0.481s
Philosopher 2 entered the room semaphore.			

The event history shows how semaphores coordinate philosophers competing for forks and help prevent deadlock during concurrent execution.

7.4 Dining Philosophers Using Monitor

Event	**5**	—	0.00	s
Philosopher 3 is thinking				
Event	**6**	—	0.42	s
Philosopher 3 is hungry and waiting				
Event	**7**	—	0.42	s
Philosopher 3 acquired both forks				
Event	**8**	—	0.42	s
Philosopher 3 started eating				
Event	**9**	—	0.59	s
Philosopher 2 is hungry and waiting				
Event	**10**	—	0.63	s
Philosopher 0 is hungry and waiting				

The event history illustrates how the monitor coordinates philosophers access to forks and ensures that neighboring philosophers do not eat simultaneously.

8.Race Conditions, Mutual Exclusion, Deadlock and Starvation

1. Race Condition

In computer programming, a race condition occurs when multiple threads use the same data without proper synchronization. The problem of race conditions can lead to erroneous results, for instance, failure to update a database. In case of Readers-Writers programs, synchronization is used to avoid conflicts between reading and writing operations.

2. Mutual Exclusion

Mutual exclusion ensures that only one process uses the resource at any given time if there is exclusive access required. If we talk about readers-writers problem, then there will be exclusive access required for the writer on the shared database. In dining philosophers problem, exclusive access would be required for the fork by one philosopher.

3. Deadlock

Deadlock occurs when a set of threads continually wait for resources held by each other. In the semaphore algorithm for dining philosophers' problem, the room semaphore manages the contention between the three philosophers. The forks are obtained together in the monitor approach to avoid deadlock.

4. Starvation

Starvation occurs when the process keeps on waiting indefinitely as the other processes gain access to the required resource. Starvation may take place in the Readers-Writers semaphore algorithm because of the continuous arrival of readers that causes starvation of writers. Starvation issue in the monitor algorithm is overcome due to the fact that writers have more priority than readers.

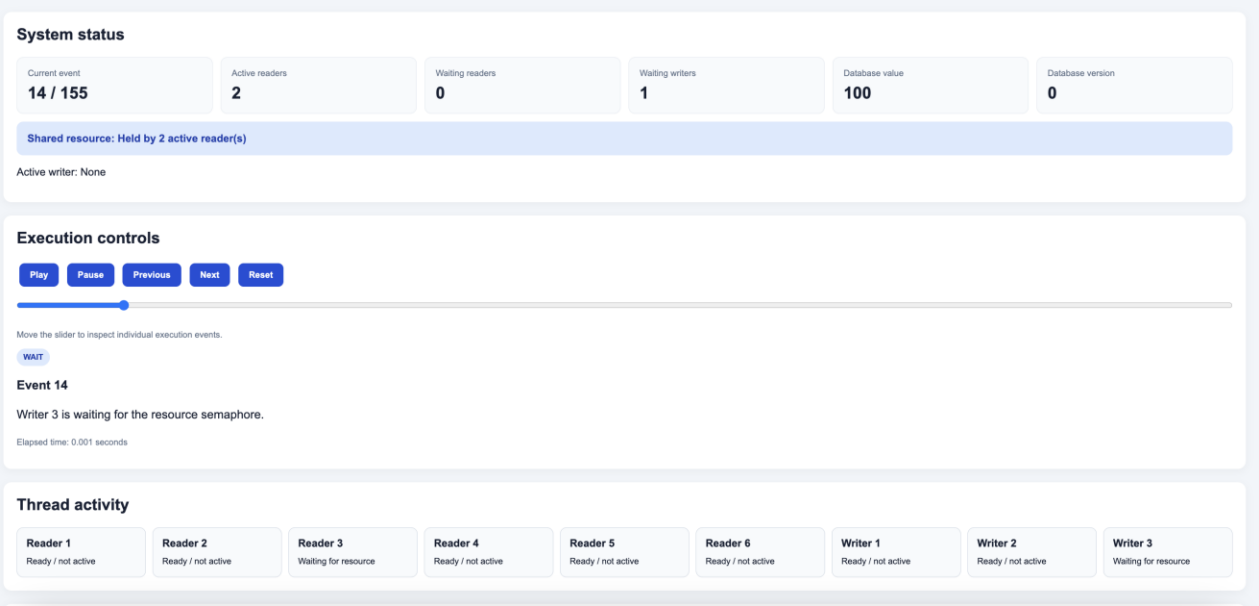
9. Semaphore vs Monitor Comparison

Aspect	Semaphore	Monitor
Structure	Uses synchronization mechanisms such as semaphores and locks.	Uses lock in conjunction with condition variables for sharing data and performing synchronization.
Resource control	The developer explicitly invokes the acquire() and release() methods.	Access control of resources is achieved through monitors and conditions.
Waiting	Waiting happens until the time when there are no semaphores available.	Condition variables are used by threads that need to wait until the condition becomes true.
Error risk	Incorrect use of acquiring and releasing leads to synchronization issues.	Encapsulation helps in synchronization and reduces errors.
Readers-Writers	Semaphore uses the reader preference technique.	Monitor uses writer preference.
Dining Philosophers	Usage of two forks semaphores separately and room semaphore to prevent deadlock.	Both forks are allocated at the same time to prevent deadlock.
Level of abstraction	Synchronization Technique of Lower Level.	Synchronization Technique of Higher Level.
Fairness	The FIFO policy is not necessarily strict.	In this case, FIFO policy is not strictly followed.

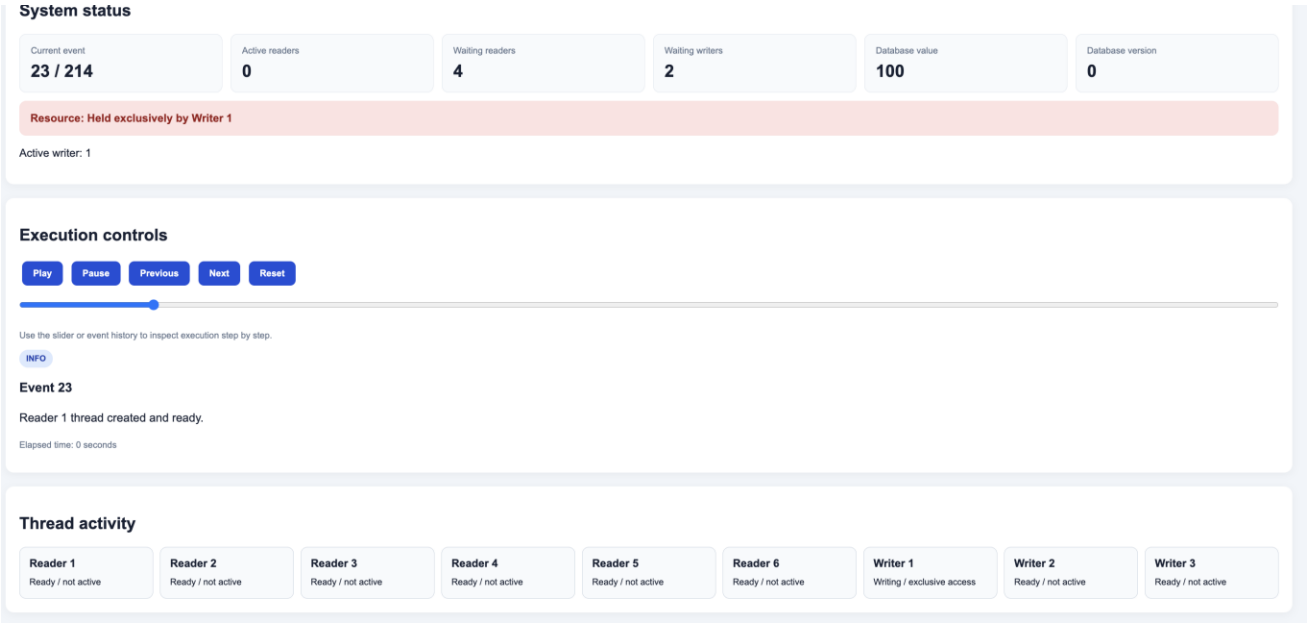
Conclusion: Semaphores provide explicit resource management whereas Monitors provide encapsulation of the shared resources and synchronization. Both of them can help prevent the race condition but their effectiveness depends on the synchronization method used.

10. Screenshots of Generated HTML Simulations

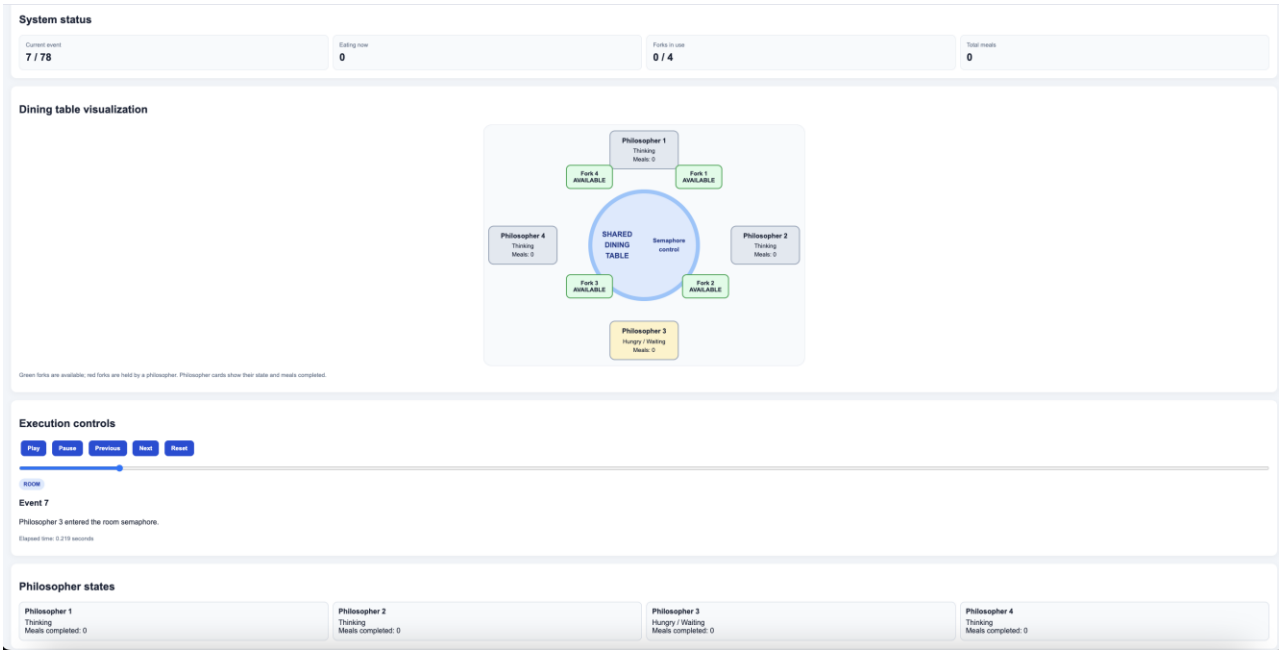
10.1 Readers-Writers Using Semaphore



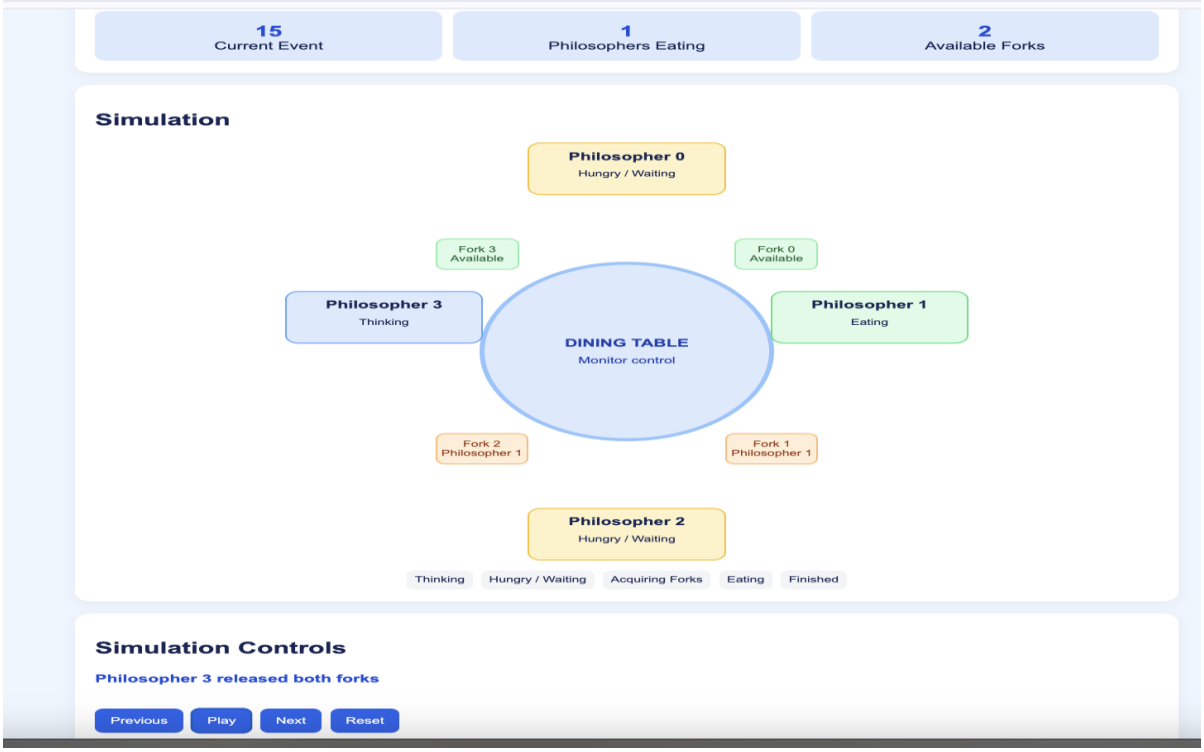
10.2 Readers-Writers Using Monitors



10.3 Dining Philosophers Using Semaphore



10.4 Dining Philosophers Using Monitor



11. AI Usage Summary

As an element of the course of Operating Systems Programming Assignment, ChatGPT (OpenAI) was utilized in order to grasp synchronization ideas, assist in writing the four programs in Python, identify and correct programming mistakes, give explanations concerning the work of the program, and develop HTML simulations.

The assignment consists of exploring the Readers-Writers and Dining Philosophers problem as well as solutions to them through semaphores and monitors.Explanation concerning mutual exclusion, race condition, deadlock avoidance, starvation, and the difference between semaphores and monitors was provided by means of ChatGPT.

HTML simulations were developed with the assistance of ChatGPT in order to demonstrate the state of threads, resource allocation, waiting condition, and actions.Code and text of the report were examined in order to deepen the understanding of the synchronization principles and mechanisms.

12. References

1.Silberschatz, A., Galvin, P.B., & Gagne, G., Operating System Concepts, 10th Edition, Wiley, 2018.

2.Downey, A.B. "The Little Book of Semaphores." Green Tea Press. URL: https://greenteapress.com/wp/semaphores/

3.Python Software Foundation, "threading - Thread-based parallelism," Python Documentation. URL: https://docs.python.org/3/library/threading.html

4.Python Software Foundation, "time - Time access and conversions," Python Documentation. URL: https://docs.python.org/3/library/time.html

5.Hoare, C.A.R., "Monitors: An Operating System Structuring Concept," Communications of the ACM, Vol. 17, No. 10, pp. 549-557, October 1974.

6.Dijkstra, E.W., "Cooperating Sequential Processes," in Programming Languages, ed. F. Genuys, Academic Press, 1968.

7.Dijkstra, E.W., "A Discipline of Programming." Englewood Cliffs, NJ: Prentice-Hall, 1976.

8.Python Software Foundation, "json - JSON encoder and decoder," Python Documentation. URL: https://docs.python.org/3/library/json.html

9.Python Software Foundation, "webbrowser - Convenient Web-browser controller," Python Documentation. URL: https://docs.python.org/3/library/webbrowser.html

10.OpenAI. “ChatGPT,” AI tool to help with synchronization concepts, Python code explanation, debugging, HTML simulation development. URL: https://chatgpt.com/

13. Academic Integrity Declaration

It confirms that I have run and understood the programs submitted. I have cited the use of any artificial intelligence source and outside sources used while doing this assignment. I can defend the synchronization methods, HTML simulation, and changes in the code made in the programs. I have made all the efforts I could make to ensure that the work I have submitted is my own work.

Name: Disha Shetty

USN: NNM24IS077

Signature: _____

Date: _____